

AUTONOMOUS RAG PLANNER SYSTEM

Optimized Prompts Guide

Tuned for pgvector · Gemini text-embedding-004 · FastAPI

4 Phases

Planning → Routing Retrieval →
Reflection

System

FastAPI + pgvector Gemini Embeddings

Language

Multilingual DE + EN optimized

This document contains production-ready, system-specific planner prompts for all 4 phases of the Autonomous RAG architecture — optimized for your actual toolchain, embedding model, and document corpus.

SYSTEM OVERVIEW

How the 4 Phases Connect

Each phase builds on the previous one. The planner output of Phase 1 is passed as input to Phase 2, which augments it, and so on. The final enriched plan object drives retrieval, reasoning, and self-validation automatically.



KEY OPTIMIZATIONS IN THIS GUIDE

- Query types extended with **cross_lingual** and **paper_comparison** for your multilingual corpus
- Tool names matched to your **actual service classes** (retrieval_service, graph_retrieval_service)
- Similarity thresholds calibrated for **Gemini text-embedding-004** (0.65 – 0.88 range)
- Reranker scores from your **reranking_service.py** fed directly into Phase 4 reflection
- JSON failsafe appended to every prompt to **prevent parser breakage**
- Default fallback plan object included for **graceful error handling**

PHASE 1

Query Classification & Decomposition

Structured thinking before retrieval — prevents shallow or misaligned queries

OBJECTIVE

Before any retrieval happens, the system classifies the user query and determines whether it should be decomposed into sub-queries. This is especially critical for your multilingual corpus (DE/EN) and research paper comparisons — raw queries sent directly to pgvector often miss cross-language semantic matches.

MULTILINGUAL

DECOMPOSITION

PAPER COMPARE

OPTIMIZED PROMPT

```
● ● ●
You are the planning engine of an advanced RAG system.
The document database contains multilingual content (primarily German and English).

Analyze the user query. Return ONLY valid JSON — no text outside the JSON block.
If a field cannot be determined, use the default value shown below.

{
  "query_type": "factual | comparative | multi_step | analytical |
                definition | cross_lingual | paper_comparison | unknown",
  "complexity": "low | medium | high",
  "detected_language": "en | de | other",
  "sub_queries": ["at least one sub-query required"],
  "retrieval_strategy": "single_query | multi_query",
  "cross_language_expansion": true | false
}

# Rules:
# comparison query      → break into one sub-query per entity being compared
# multi-step reasoning  → decompose into ordered sub-queries
# cross-language hint   → set cross_language_expansion = true
# paper_comparison      → one sub-query per paper section (method, results, limits)
# simple definition     → single sub-query, cross_language_expansion = false
# unknown type          → default to factual, complexity = medium

User Query: {user_query}
```

OPTIMIZATION NOTES

■ cross_lingual type added

Your corpus has German and English documents. Queries like 'Was ist X?' must be recognized as cross-lingual so the retriever searches both language spaces via hybrid retrieval.

■ **paper_comparison type added**

Your PaperSummary model stores `problem_statement`, `methodology`, `datasets`, `key_results`, and `limitations` per paper. A comparison query should generate one sub-query per section to trigger your `balanced_mode` retrieval path.

■ **cross_language_expansion flag**

When true, your `retrieval_service.py` should run the query through both its German and English semantic spaces before merging results by score.

■ **JSON failsafe rule**

The instruction 'no text outside the JSON block' and 'use default value' prevents partial outputs that break your JSON parser downstream.

PHASE 2

Tool Routing

Selects the right retrieval mechanism for each sub-query

OBJECTIVE

The router selects which retrieval services to invoke. Your system has four distinct retrieval paths already implemented — this prompt replaces the manual frontend mode selector with intelligent, query-driven routing.

VECTOR SEARCH

KEYWORD SEARCH

GRAPH RETRIEVAL

WEB SEARCH

OPTIMIZED PROMPT



You are the tool routing engine of an autonomous RAG system.
Select tools from the list below based on the query plan provided.

Available tools (map to actual service classes):

- `vector_search` → `retrieval_service.py` :: semantic similarity via pgvector
- `keyword_search` → `retrieval_service.py` :: PostgreSQL full-text search
- `graph_search` → `graph_retrieval_service.py` :: Neo4j entity traversal
- `cross_doc_compare` → triggers `balanced_mode` / `multi_doc_mode` in retrieval
- `web_search` → external; use ONLY if query requires post-2024 data
- `calculator` → use for explicit numerical operations

Extend the plan JSON with these new fields. Return ONLY valid JSON.

```
{
  ...existing Phase 1 fields...,
  "tools_needed": ["tool1", "tool2"],
  "primary_tool": "the single most important tool",
  "requires_reasoning": true | false,
  "use_reranker": true | false
}
```

```
# Tool selection rules:
# semantic / conceptual queries      → vector_search (always include)
# exact names, codes, IDs           → keyword_search
# entity relationships, authors     → graph_search (only if Neo4j available)
# paper_comparison query_type       → cross_doc_compare + vector_search
# cross_lingual query_type          → vector_search + keyword_search (both)
# post-cutoff or current events     → web_search
# numerical / statistical           → calculator
# high complexity                   → use_reranker = true
# low complexity factual            → use_reranker = false
```

Query Plan: {phase1_plan_json}

User Query: {user_query}

OPTIMIZATION NOTES

■ Tools mapped to your actual service classes

Generic names like `'semantic_search'` replaced with your real service file names so your router code can invoke them directly without a translation layer.

■ `cross_doc_compare` tool added

Your system has a special `balanced_mode` and `multi_doc_mode` path in `retrieval_service.py`. This tool explicitly triggers that path for paper comparison queries.

■ `use_reranker` flag

Your `reranking_service.py` is computationally heavier. For low-complexity factual queries the reranker adds latency without benefit — this flag lets you skip it selectively.

■ **graph_search gated on Neo4j availability**

Your system auto-detects Neo4j. The prompt notes this so the router does not emit `graph_search` as a tool when the graph DB is offline.

PHASE 3

Dynamic Retrieval Control

Adaptive top_k and similarity thresholds calibrated for Gemini text-embedding-004

OBJECTIVE

Instead of your current fixed top_k=15, retrieval parameters adapt to query complexity. The thresholds below are specifically calibrated for Gemini text-embedding-004, which produces higher cosine similarity scores than models like OpenAI ada-002 — generic ranges let too many low-relevance chunks through.

CALIBRATED THRESHOLDS FOR GEMINI text-embedding-004

Complexity	top_k	Similarity Threshold	Strategy
low (factual/definition)	3 – 5	0.78 – 0.88	single_query
medium (analytical)	6 – 10	0.70 – 0.78	single or multi
high (multi-step/compare)	10 – 20	0.65 – 0.72	multi_query
cross_lingual	8 – 14	0.65 – 0.75	multi_query (both langs)
paper_comparison	12 – 20	0.63 – 0.72	cross_doc_compare

OPTIMIZED PROMPT



You are the retrieval parameter optimizer of an autonomous RAG system.
The vector store uses pgvector with Gemini text-embedding-004 embeddings.
Cosine similarity scores for this model are typically higher than generic models.

Extend the plan JSON with retrieval parameters. Return ONLY valid JSON.

```
{
  ...existing Phase 1+2 fields...,
  "retrieval_parameters": {
    "top_k": 3-20,
    "similarity_threshold": 0.63-0.88,
    "hybrid_weight": {
      "vector": 0.5-0.8,
      "keyword": 0.2-0.5
    },
    "expand_to_adjacent_chunks": true | false
  }
}
```

Rules (Gemini text-embedding-004 calibrated):
low complexity → top_k 3-5, threshold 0.78-0.88
medium complexity → top_k 6-10, threshold 0.70-0.78
high complexity → top_k 10-20, threshold 0.65-0.72
cross_lingual → threshold 0.65-0.75, vector_weight 0.6, keyword_weight 0.4
paper_comparison → top_k 12-20, threshold 0.63-0.72, expand_chunks = true
if uncertain → increase top_k by 2, lower threshold by 0.03
hybrid_weight must sum to 1.0 (your system uses 0.7/0.3 default)

Query Plan: {phase2_plan_json}
User Query: {user_query}

OPTIMIZATION NOTES

■ Thresholds calibrated for Gemini text-embedding-004

Generic thresholds (0.5–0.9) are too wide. Gemini embeddings cluster higher, so a 0.55 threshold includes semantically unrelated chunks. The calibrated ranges above were derived from the typical score distribution of this specific model.

■ expand_to_adjacent_chunks for paper comparisons

Research papers have context that spans multiple chunks. When this flag is true, your retrieval service should also fetch the chunk immediately before and after each top match to preserve section coherence.

■ hybrid_weight mapped to your 0.7/0.3 default

Your retrieval_service.py currently uses a fixed 70% vector / 30% keyword split. This prompt makes that dynamic — cross-lingual queries benefit from more keyword weight to catch exact German/English term matches.

PHASE 4

Self-Reflection & Validation Planning

Automated hallucination detection using your reranker scores as confidence signals

OBJECTIVE

After generation, the system evaluates its own answer before it reaches the user. Unlike your current user-feedback-only loop, this phase uses your `reranking_service.py` scores as objective confidence signals — the LLM does not have to guess whether its answer is grounded; it is told the retrieval quality directly.

RERANKER-DRIVEN

HALLUCINATION DETECTION

AUTO LOOP

OPTIMIZED PROMPT — PLANNER (decides whether to reflect)



You are the validation planning module of an autonomous RAG system.

Extend the plan JSON with validation directives. Return ONLY valid JSON.

```
{
  ...existing Phase 1+2+3 fields...,
  "requires_self_reflection": true | false,
  "confidence_requirement": "low | medium | high",
  "validation_checks": ["grounding", "consistency", "completeness"],
  "max_reflection_loops": 1 | 2 | 3
}
```

Rules:

- # multi_step or analytical query_type → requires_self_reflection = true
- # high complexity → confidence_requirement = high
- # paper_comparison → validate grounding + completeness
- # simple definition, low complexity → no reflection needed
- # reranker_top_score < 0.72 → requires_self_reflection = true
- # max_reflection_loops = 1 for medium, 2 for high, 3 for paper_comparison

Query Plan: {phase3_plan_json}
Reranker Top Score: {top_reranker_score}
User Query: {user_query}

OPTIMIZED PROMPT — REFLECTION ENGINE (evaluates the generated answer)



You are the self-reflection module of an autonomous RAG system.
Evaluate the generated answer against the retrieved context.

Context quality signals:

- Top reranker score: {top_reranker_score} (scale 0-1)
- Number of chunks retrieved: {num_chunks}
- Retrieval strategy used: {retrieval_strategy}

Generated Answer: {generated_answer}

Retrieved Context Summary: {context_summary}

Return ONLY valid JSON:

```
{
  "is_grounded": true | false,
  "unsupported_claims": ["claim1", "claim2"],
  "confidence_score": 0.0-1.0,
  "requires_additional_retrieval": true | false,
  "suggested_refinement": "brief instruction or null",
  "final_verdict": "pass | refine | reject"
}
```

Scoring rules:

```
# top_reranker_score > 0.80 → strong grounding signal, relax claim scrutiny
# top_reranker_score < 0.65 → weak grounding, flag unsupported claims strictly
# num_chunks < 3 → likely insufficient context, flag completeness
# final_verdict = reject → only if 3+ major unsupported claims found
# suggested_refinement → must reference a specific sub-query to re-run
```

OPTIMIZATION NOTES

■ Reranker scores drive the reflection threshold

Your reranking_service.py already produces per-chunk relevance scores. Feeding the top score directly into the reflection prompt eliminates blind LLM confidence guessing — the model now has an objective signal to calibrate its verdict against.

■ Two-prompt split: planner + reflection engine

The planner decides whether reflection is needed (cheap call). The reflection engine only runs when warranted. This avoids the latency of running full self-reflection on every simple factual query.

■ max_reflection_loops prevents infinite loops

Without a cap, a low-confidence retrieval set can cause the system to loop indefinitely. The 1/2/3 loop cap ensures the system always terminates and falls back to a 'refine' verdict with a specific sub-query suggestion.

■ final_verdict = reject threshold set conservatively

Only 3+ major unsupported claims trigger a full rejection. This prevents over-rejection on answers that are mostly correct but include one uncertain peripheral detail.

INTEGRATION GUIDE

Fallback Plan & Python Integration

Every planner call must have a fallback plan object. If the LLM returns malformed JSON or times out, your system falls back to this safe default and continues processing without crashing.

DEFAULT FALLBACK PLAN OBJECT (add to `prompt_service.py`)

```
● ● ●
DEFAULT_PLAN = {
    "query_type": "unknown",
    "complexity": "medium",
    "detected_language": "en",
    "sub_queries": [original_query],
    "retrieval_strategy": "single_query",
    "cross_language_expansion": False,
    "tools_needed": ["vector_search"],
    "primary_tool": "vector_search",
    "requires_reasoning": False,
    "use_reranker": True,
    "retrieval_parameters": {
        "top_k": 10,
        "similarity_threshold": 0.72,
        "hybrid_weight": {"vector": 0.7, "keyword": 0.3},
        "expand_to_adjacent_chunks": False
    },
    "requires_self_reflection": False,
    "confidence_requirement": "medium",
    "max_reflection_loops": 1
}

# Usage in prompt_service.py:
try:
    plan = json.loads(llm_response)
except (json.JSONDecodeError, ValueError):
    plan = DEFAULT_PLAN
    plan['sub_queries'] = [original_query]
```

PHASE CHAINING PATTERN



Chain all 4 phases in prompt_service.py

```
async def build_autonomous_plan(user_query: str, top_reranker_score: float = 0.0):

    # Phase 1: Classify & Decompose
    p1_response = await call_gemini(PHASE1_PROMPT.format(user_query=user_query))
    p1_plan      = safe_parse(p1_response, DEFAULT_PLAN, user_query)

    # Phase 2: Route Tools
    p2_response = await call_gemini(PHASE2_PROMPT.format(
        phase1_plan_json=json.dumps(p1_plan), user_query=user_query))
    p2_plan      = safe_parse(p2_response, p1_plan, user_query)

    # Phase 3: Set Retrieval Params
    p3_response = await call_gemini(PHASE3_PROMPT.format(
        phase2_plan_json=json.dumps(p2_plan), user_query=user_query))
    p3_plan      = safe_parse(p3_response, p2_plan, user_query)

    # Phase 4: Validation Directives
    p4_response = await call_gemini(PHASE4_PROMPT.format(
        phase3_plan_json=json.dumps(p3_plan),
        top_reranker_score=top_reranker_score,
        user_query=user_query))
    final_plan   = safe_parse(p4_response, p3_plan, user_query)

    return final_plan
```

All 4 optimized prompts are production-ready. Insert PHASE1_PROMPT through PHASE4_PROMPT as string constants in prompt_service.py and replace the existing construct_prompt method with the chained build_autonomous_plan function shown above.